

"""

## AI-Powered Predictive Maintenance using Digital Twin

Team: Tech Titans | PICT Pune | April 2026

Members: Anuja Naik, Harsh Takalkar, Aditya Patil, Aditya Gaidhane

This file contains:

1. Data generation (simulated industrial sensor data)
2. Autoencoder model (PyTorch) - Digital Twin core
3. Training pipeline
4. Health Index computation
5. RUL (Remaining Useful Life) prediction
6. What-If simulation engine
7. Decision engine with recommendations

"""

```
import numpy as np
import pandas as pd
import json
from dataclasses import dataclass, field
from typing import List, Dict, Tuple, Optional
import math
```

```
# _____
```

```
# 1. SENSOR DATA GENERATOR
```

```
# _____
```

```
class SensorDataGenerator:
```

```
    """Simulates industrial machine sensor data for training and inference."""
```

```
    def __init__(self, seed: int = 42):
```

```
        np.random.seed(seed)
```

```

def generate_normal_data(self, n_samples: int = 1000) -> np.ndarray:
    """
    Generate 'normal' operating data for autoencoder training.
    Features: [temperature_norm, load_norm, vibration_norm]
    All values normalized to [0, 1].
    """
    # Normal operating ranges
    temp = np.random.normal(0.45, 0.08, n_samples) # ~65°C normalized (120°C max)
    load = np.random.normal(0.50, 0.10, n_samples) # ~50% load
    vib = np.random.normal(0.25, 0.05, n_samples) # ~2.5mm/s normalized (10mm/s max)

    # Clip to [0.05, 0.95] to avoid boundary effects
    temp = np.clip(temp, 0.05, 0.95)
    load = np.clip(load, 0.05, 0.95)
    vib = np.clip(vib, 0.05, 0.95)

    data = np.column_stack([temp, load, vib])
    return data.astype(np.float32)

```

```

def generate_anomalous_data(self, n_samples: int = 100) -> np.ndarray:
    """
    Generate anomalous data (overheating, overload, high vibration).
    Used ONLY for evaluation — NOT used in training.
    """
    # Anomaly type 1: Overheating
    temp_anom = np.random.uniform(0.80, 1.0, n_samples // 3)
    load_anom = np.random.normal(0.50, 0.08, n_samples // 3)
    vib_anom = np.random.normal(0.25, 0.05, n_samples // 3)
    overheating = np.column_stack([temp_anom, load_anom, vib_anom])

```

```

# Anomaly type 2: Overload
t2 = np.random.normal(0.45, 0.08, n_samples // 3)
l2 = np.random.uniform(0.85, 1.0, n_samples // 3)
v2 = np.random.uniform(0.60, 0.90, n_samples // 3)
overload = np.column_stack([t2, l2, v2])

# Anomaly type 3: Bearing wear (high vibration)
t3 = np.random.normal(0.45, 0.08, n_samples // 3)
l3 = np.random.normal(0.50, 0.10, n_samples // 3)
v3 = np.random.uniform(0.75, 1.0, n_samples // 3)
bearing_wear = np.column_stack([t3, l3, v3])

data = np.vstack([overheating, overload, bearing_wear])
return np.clip(data, 0, 1).astype(np.float32)

```

```

# _____
# 2. AUTOENCODER (NumPy implementation)
# Architecture: 3 → 8 → 4 → 2 → 4 → 8 → 3
# _____

```

```

class AutoencoderNumPy:

```

```

    """

```

```

    Lightweight autoencoder trained on normal machine data.

```

```

    Unsupervised anomaly detection via reconstruction error (MSE).

```

```

    During inference:

```

- Normal input → low reconstruction error → high health
- Anomalous input → high reconstruction error → low health / anomaly flag

```

    """

```

```

def __init__(self, input_dim: int = 3, lr: float = 0.01):
    self.input_dim = input_dim
    self.lr = lr

    # Xavier initialization
    def xavier(fan_in, fan_out):
        limit = math.sqrt(6.0 / (fan_in + fan_out))
        return np.random.uniform(-limit, limit, (fan_out, fan_in))

    # Encoder weights & biases: 3→8→4→2
    self.W1 = xavier(3, 8); self.b1 = np.zeros(8)
    self.W2 = xavier(8, 4); self.b2 = np.zeros(4)
    self.W3 = xavier(4, 2); self.b3 = np.zeros(2)

    # Decoder weights & biases: 2→4→8→3
    self.W4 = xavier(2, 4); self.b4 = np.zeros(4)
    self.W5 = xavier(4, 8); self.b5 = np.zeros(8)
    self.W6 = xavier(8, 3); self.b6 = np.zeros(3)

    # Activations
    @staticmethod
    def relu(x): return np.maximum(0, x)

    @staticmethod
    def relu_deriv(x): return (x > 0).astype(float)

    @staticmethod
    def sigmoid(x): return 1 / (1 + np.exp(-np.clip(x, -500, 500)))

    @staticmethod

```

```

def sigmoid_deriv(x):

    s = 1 / (1 + np.exp(-np.clip(x, -500, 500)))

    return s * (1 - s)


def forward(self, x: np.ndarray) -> Dict:

    """Forward pass — encoder → bottleneck → decoder."""

    # Encoder

    z1 = x @ self.W1.T + self.b1;    a1 = self.relu(z1)

    z2 = a1 @ self.W2.T + self.b2;    a2 = self.relu(z2)

    z3 = a2 @ self.W3.T + self.b3;    latent = self.relu(z3) # 2D latent space


    # Decoder

    z4 = latent @ self.W4.T + self.b4; a4 = self.relu(z4)

    z5 = a4 @ self.W5.T + self.b5;    a5 = self.relu(z5)

    z6 = a5 @ self.W6.T + self.b6;    recon = self.sigmoid(z6) # [0,1] output


    return {

        'z1': z1, 'a1': a1, 'z2': z2, 'a2': a2,

        'z3': z3, 'latent': latent,

        'z4': z4, 'a4': a4, 'z5': z5, 'a5': a5,

        'z6': z6, 'recon': recon

    }


def compute_loss(self, x: np.ndarray, recon: np.ndarray) -> float:

    """Mean Squared Error loss."""

    return float(np.mean((x - recon) ** 2))


def backward(self, x: np.ndarray, cache: Dict):

    """Backpropagation — compute gradients."""

    recon = cache['recon']

    N = x.shape[0] if x.ndim > 1 else 1

```

```

# Output layer gradient (MSE + sigmoid)
dL_drecon = 2 * (recon - x) / self.input_dim
dz6 = dL_drecon * self.sigmoid_deriv(cache['z6'])

dW6 = dz6.T @ cache['a5'] / N;    db6 = dz6.mean(axis=0) if x.ndim > 1 else dz6
da5 = dz6 @ self.W6

dz5 = da5 * self.relu_deriv(cache['z5'])
dW5 = dz5.T @ cache['a4'] / N;    db5 = dz5.mean(axis=0) if x.ndim > 1 else dz5
da4 = dz5 @ self.W5

dz4 = da4 * self.relu_deriv(cache['z4'])
dW4 = dz4.T @ cache['latent'] / N; db4 = dz4.mean(axis=0) if x.ndim > 1 else dz4
dlatent = dz4 @ self.W4

dz3 = dlatent * self.relu_deriv(cache['z3'])
dW3 = dz3.T @ cache['a2'] / N;    db3 = dz3.mean(axis=0) if x.ndim > 1 else dz3
da2 = dz3 @ self.W3

dz2 = da2 * self.relu_deriv(cache['z2'])
dW2 = dz2.T @ cache['a1'] / N;    db2 = dz2.mean(axis=0) if x.ndim > 1 else dz2
da1 = dz2 @ self.W2

dz1 = da1 * self.relu_deriv(cache['z1'])
dW1 = dz1.T @ x / N;             db1 = dz1.mean(axis=0) if x.ndim > 1 else dz1

return {
    'dW1': dW1, 'db1': db1, 'dW2': dW2, 'db2': db2,
    'dW3': dW3, 'db3': db3, 'dW4': dW4, 'db4': db4,
    'dW5': dW5, 'db5': db5, 'dW6': dW6, 'db6': db6,

```

```
}
```

```
def update_weights(self, grads: Dict):
```

```
    """Gradient descent weight update."""
```

```
    self.W1 -= self.lr * grads['dW1']
```

```
    self.b1 -= self.lr * grads['db1']
```

```
    self.W2 -= self.lr * grads['dW2']
```

```
    self.b2 -= self.lr * grads['db2']
```

```
    self.W3 -= self.lr * grads['dW3']
```

```
    self.b3 -= self.lr * grads['db3']
```

```
    self.W4 -= self.lr * grads['dW4']
```

```
    self.b4 -= self.lr * grads['db4']
```

```
    self.W5 -= self.lr * grads['dW5']
```

```
    self.b5 -= self.lr * grads['db5']
```

```
    self.W6 -= self.lr * grads['dW6']
```

```
    self.b6 -= self.lr * grads['db6']
```

```
def train(self, data: np.ndarray, epochs: int = 100,
```

```
        batch_size: int = 32, verbose: bool = True) -> List[float]:
```

```
    """Train autoencoder on NORMAL data only."""
```

```
    losses = []
```

```
    n = len(data)
```

```
    print(f"\n{'='*50}")
```

```
    print(f"Training Autoencoder (Digital Twin Core)")
```

```
    print(f"Architecture: {self.input_dim}→8→4→2→4→8→{self.input_dim}")
```

```
    print(f"Samples: {n} | Epochs: {epochs} | LR: {self.lr}")
```

```
    print(f"{'='*50}")
```

```
    for epoch in range(epochs):
```

```
        idx = np.random.permutation(n)
```

```
epoch_loss = 0
```

```
for i in range(0, n, batch_size):
```

```
    batch = data[idx[i:i+batch_size]]
```

```
    cache = self.forward(batch)
```

```
    loss = self.compute_loss(batch, cache['recon'])
```

```
    grads = self.backward(batch, cache)
```

```
    self.update_weights(grads)
```

```
    epoch_loss += loss
```

```
avg_loss = epoch_loss / (n // batch_size)
```

```
losses.append(avg_loss)
```

```
if verbose and (epoch % 10 == 0 or epoch == epochs - 1):
```

```
    print(f"Epoch {epoch+1:3d}/{epochs} | Loss (MSE): {avg_loss:.5f}")
```

```
print(f"\n✓ Training complete. Final MSE: {losses[-1]:.5f}")
```

```
return losses
```

```
def predict_mse(self, x: np.ndarray) -> float:
```

```
    """Run forward pass and return reconstruction MSE."""
```

```
    if x.ndim == 1:
```

```
        x = x.reshape(1, -1)
```

```
    cache = self.forward(x)
```

```
    return self.compute_loss(x, cache['recon'])
```

```
def get_latent(self, x: np.ndarray) -> np.ndarray:
```

```
    """Return 2D latent space encoding."""
```

```
    if x.ndim == 1:
```

```
        x = x.reshape(1, -1)
```

```
    cache = self.forward(x)
```



```
return cache['latent']
```

```
# _____
```

```
# 3. HEALTH INDEX & RUL COMPUTATION
```

```
# _____
```

```
class HealthComputer:
```

```
    """
```

```
    Converts autoencoder MSE to Health Index and Remaining Useful Life.
```

```
    """
```

```
ANOMALY_THRESHOLD = 0.15  # MSE above this → anomaly
```

```
WARN_THRESHOLD = 0.08  # MSE above this → warning
```

```
CRITICAL_HEALTH = 20  # % below this → critical maintenance
```

```
@staticmethod
```

```
def mse_to_health(mse: float) -> int:
```

```
    """
```

```
    Map reconstruction error to health score (0–100%).
```

```
    Normal operation: MSE < 0.05 → Health ~95-100%
```

```
    Anomalous:    MSE > 0.15 → Health < 25%
```

```
    """
```

```
    normalized = min(mse / 0.20, 1.0)
```

```
    return max(0, round((1 - normalized) * 100))
```

```
@staticmethod
```

```
def compute_rul(health: int, temp: float, load: float, vib: float) -> int:
```

```
    """
```

```
    Estimate Remaining Useful Life in days using a stress-based degradation model.
```

Stress score combines:

- Temperature contribution: 40%
- Load contribution: 35%
- Vibration contribution: 25%

$$RUL = (health - critical\_threshold) / degradation\_rate \times correction\_factor$$

"""

temp\_norm = (temp - 20) / 100 # normalize to [0,1]

load\_norm = load / 100

vib\_norm = vib / 10

stress = (temp\_norm \* 0.40) + (load\_norm \* 0.35) + (vib\_norm \* 0.25)

degradation\_rate = 0.1 + stress \* 0.9 # faster degradation under stress

health\_above\_critical = max(0, health - HealthComputer.CRITICAL\_HEALTH)

rul = round(health\_above\_critical / max(degradation\_rate, 0.01) \* 1.5)

return max(0, rul)

@staticmethod

def get\_status(mse: float, health: int) -> Dict:

"""Return status label and recommended action."""

if mse > HealthComputer.ANOMALY\_THRESHOLD or health < 30:

return {

    'status': 'CRITICAL',

    'color': 'red',

    'action': 'Immediate maintenance required. Shutdown recommended.'

}

elif mse > HealthComputer.WARN\_THRESHOLD or health < 60:

return {

    'status': 'WARNING',

    'color': 'amber',

```

        'action': 'Schedule preventive maintenance within 48 hours.'
    }
else:
    return {
        'status': 'NORMAL',
        'color': 'green',
        'action': 'Continue monitoring. No intervention needed.'
    }

```

```

# -----
# 4. WHAT-IF SIMULATION ENGINE
# -----

```

```

class WhatIfSimulator:
    """
    Interactive What-If scenario engine.
    Takes user-specified sensor parameters, runs them through the
    trained digital twin, and returns predicted outcomes.
    """

    def __init__(self, model: AutoencoderNumPy):
        self.model = model
        self.health_comp = HealthComputer()

    def simulate(self, temp_c: float, load_pct: float, vib_mms: float) -> Dict:
        """
        Run a what-if scenario.

        Args:
            temp_c: Temperature in Celsius (20–120)

```

load\_pct: Load percentage (0–100)

vib\_mms: Vibration in mm/s (0–10)

Returns:

Dictionary with: mse, health, rul, status, recommendations

"""

# Normalize inputs

input\_vec = np.array([

(temp\_c - 20) / 100,

load\_pct / 100,

vib\_mms / 10

], dtype=np.float32)

# Forward pass through digital twin

mse = self.model.predict\_mse(input\_vec)

# Compute health and RUL

health = self.health\_comp.mse\_to\_health(mse)

rul = self.health\_comp.compute\_rul(health, temp\_c, load\_pct, vib\_mms)

status = self.health\_comp.get\_status(mse, health)

# Generate recommendations

recs = self.\_generate\_recommendations(temp\_c, load\_pct, vib\_mms, mse, health)

return {

'inputs': {

'temperature\_C': round(temp\_c, 1),

'load\_pct': round(load\_pct, 1),

'vibration\_mms': round(vib\_mms, 2)

},

'outputs': {

```

'reconstruction_mse': round(mse, 5),
'health_index_pct': health,
'rul_days': rul,
'anomaly_detected': mse > HealthComputer.ANOMALY_THRESHOLD,
},
'status': status,
'recommendations': recs
}

```

```

def _generate_recommendations(self, temp, load, vib, mse, health) -> List[str]:
    recs = []
    if temp > 90:
        recs.append(f"Temperature {temp:.1f}°C exceeds safe limit (90°C). Improve cooling.")
    if load > 80:
        recs.append(f"Load {load:.1f}% is in overload zone. Reduce workload by {load-70:.0f}%.")
    if vib > 7:
        recs.append(f"Vibration {vib:.1f}mm/s is critical. Inspect bearings and mountings.")
    if mse > HealthComputer.ANOMALY_THRESHOLD:
        recs.append(f"Anomaly detected (MSE={mse:.4f}). Pattern deviates from normal operation.")
    if health < 40:
        recs.append(f"Health index {health}% is critically low. Schedule immediate maintenance.")
    if not recs:
        recs.append("All parameters within nominal operating range. No action required.")
    return recs

```

```

def find_optimal_parameters(self, n_trials: int = 200) -> Dict:

```

```

    """

```

```

    Simple random search to find parameter settings that maximize health.

```

```

"""

best = {'health': 0}
for _ in range(n_trials):
    t = np.random.uniform(20, 80)
    l = np.random.uniform(10, 70)
    v = np.random.uniform(0.5, 5.0)
    result = self.simulate(t, l, v)
    if result['outputs']['health_index_pct'] > best['health']:
        best = {
            'health': result['outputs']['health_index_pct'],
            'temperature_C': round(t, 1),
            'load_pct': round(l, 1),
            'vibration_mms': round(v, 2),
            'rul_days': result['outputs']['rul_days']
        }
return best

```

```

# -----
# 5. DECISION ENGINE
# -----

```

```

class DecisionEngine:
    """
    Closed-loop decision engine: monitors health trend and
    triggers automated maintenance recommendations.
    """

    def __init__(self, window: int = 10):
        self.health_history: List[int] = []
        self.window = window

```

```
def update(self, health: int) -> Optional[str]:
```

```
    """Add new health reading and check for alerts."""
```

```
    self.health_history.append(health)
```

```
    if len(self.health_history) > 50:
```

```
        self.health_history.pop(0)
```

```
    return self._check_trend()
```

```
def _check_trend(self) -> Optional[str]:
```

```
    """Detect degradation trend using sliding window."""
```

```
    if len(self.health_history) < self.window:
```

```
        return None
```

```
    window = self.health_history[-self.window:]
```

```
    slope = np.polyfit(range(self.window), window, 1)[0]
```

```
    if slope < -3.0:
```

```
        return f"⚠️ RAPID DEGRADATION: Health declining {abs(slope):.1f}% per cycle.  
Immediate inspection."
```

```
    elif slope < -1.0:
```

```
        return f"📉 GRADUAL DEGRADATION: Health declining {abs(slope):.1f}% per  
cycle. Schedule maintenance."
```

```
    elif window[-1] < HealthComputer.CRITICAL_HEALTH:
```

```
        return "🔴 CRITICAL HEALTH THRESHOLD REACHED. Shutdown recommended."
```

```
    return None
```

```
# _____
```

```
# 6. MAIN — DEMO PIPELINE
```

```
# _____
```

```
def main():
```

```
    print("\n" + "="*60)
```

```

print(" AI-Powered Predictive Maintenance using Digital Twin")
print(" Team: Tech Titans | PICT Pune | April 2026")
print("="*60)

# Step 1: Generate training data (normal operation only)
gen = SensorDataGenerator()
normal_data = gen.generate_normal_data(n_samples=800)
anomaly_data = gen.generate_anomalous_data(n_samples=150)

print(f"\n[DATA] Normal samples: {len(normal_data)} | Anomaly samples (eval only): {len(anomaly_data)}")
print(f"[DATA] Features: [temperature_norm, load_norm, vibration_norm]")

# Step 2: Train autoencoder on NORMAL data only
model = AutoencoderNumPy(input_dim=3, lr=0.005)
losses = model.train(normal_data, epochs=80, batch_size=32)

# Step 3: Evaluate on normal vs anomalous
normal_mses = [model.predict_mse(x) for x in normal_data[:50]]
anomaly_mses = [model.predict_mse(x) for x in anomaly_data[:50]]

print(f"\n[EVAL] Normal MSE — Mean: {np.mean(normal_mses):.4f} | Max: {np.max(normal_mses):.4f}")
print(f"[EVAL] Anomalous MSE — Mean: {np.mean(anomaly_mses):.4f} | Max: {np.max(anomaly_mses):.4f}")

threshold = np.percentile(normal_mses, 95)
correct = sum(m > threshold for m in anomaly_mses)
accuracy = correct / len(anomaly_mses) * 100

print(f"[EVAL] Anomaly detection accuracy: {accuracy:.1f}% (threshold={threshold:.4f})")

```



```
# Step 4: What-If Simulation
```

```
simulator = WhatIfSimulator(model)
```

```
print("\n" + "-"*60)
```

```
print("WHAT-IF SIMULATION SCENARIOS")
```

```
print("-"*60)
```

```
scenarios = [
```

```
    ("Normal operation", 65.0, 50.0, 2.5),
```

```
    ("High temperature", 100.0, 50.0, 2.5),
```

```
    ("Overload", 65.0, 90.0, 2.5),
```

```
    ("High vibration", 65.0, 50.0, 8.5),
```

```
    ("Critical combined", 105.0, 92.0, 8.8),
```

```
]
```

```
for name, temp, load, vib in scenarios:
```

```
    result = simulator.simulate(temp, load, vib)
```

```
    o = result['outputs']
```

```
    s = result['status']
```

```
    print(f"\n Scenario: {name}")
```

```
    print(f"  Input: Temp={temp}°C | Load={load}% | Vibration={vib}mm/s")
```

```
    print(f"      Output: Health={o['health_index_pct']}% | RUL={o['rul_days']}d |  
MSE={o['reconstruction_mse']:.4f}")
```

```
    print(f"  Status: [{s['status']}] {s['action']}")
```

```
    for rec in result['recommendations']:
```

```
        print(f"    → {rec}")
```

```
# Step 5: Find optimal parameters
```

```
print("\n" + "-"*60)
```

```
print("OPTIMAL PARAMETER SEARCH (Random Search, n=200)")
```

```
print("-"*60)
```

```

optimal = simulator.find_optimal_parameters(n_trials=200)
print(f" Best parameters found:")
print(f"  Temperature: {optimal['temperature_C']}°C")
print(f"  Load:      {optimal['load_pct']}%")
print(f"  Vibration:  {optimal['vibration_mms']} mm/s")
print(f"  Health Index: {optimal['health']}%")
print(f"  RUL:      {optimal['rul_days']} days")

```

# Step 6: Trend-based decision engine demo

```

print("\n" + "-"*60)
print("DECISION ENGINE — DEGRADATION TREND DETECTION")
print("-"*60)
engine = DecisionEngine(window=5)
# Simulate degrading health
for h in [95, 88, 79, 68, 54, 41, 28, 18]:
    alert = engine.update(h)
    print(f" Health={h:3d}% → {alert if alert else 'No alert'}")

```

```

print("\n✓ Pipeline complete. Dashboard:
predictive_maintenance_dashboard.html\n")

```

```

if __name__ == "__main__":
    main()

```